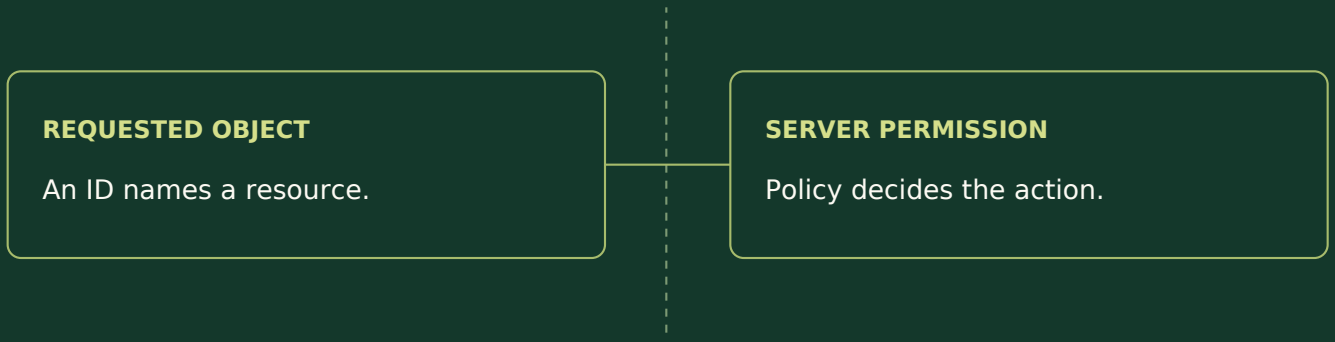
OBJECT PERMISSIONS / TRUST BOUNDARIES / SECURE DESIGN

IDOR Explained

Understand object permissions, prove access-control failures, and build safer APIs



Understand the permission before the vulnerability.
Find the failure. Prove the effect. Repair the policy.
Practice with dummy accounts on your own computer.

Haroon Shahid

First edition / Version 1.0 / 27 October 2026

Before you begin

IDOR Explained

Understand object permissions, prove access-control failures, and build safer APIs

By Haroon Shahid. First edition, version 1.0. 27 October 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework, cloud, database, and protocol guidance is version-sensitive. The local lab uses loopback HTTP, controlled accounts, dummy records, and in-memory policies. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 60 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Harbor Desk lab.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. Data, objects, and references 7

02. How a web request reaches an object 9

03. Identity, authentication, and sessions 11

Sources and further reading 13

Introduction

The ID is the address, not the permission

Imagine a support ticket with a private note. Alice can open it. Bob can log in too, but the ticket was never shared with him. Bob changes the ticket reference in his request. The server returns Alice’s private note.

The interesting part is not that Bob changed a string. It is that the server used the string to select a resource and failed to enforce the relationship that should govern access. This book teaches you to see that missing decision, prove its actual effect, and repair it across every relevant path.

You can start without knowing HTTP, databases, or access-control terminology. The first ten chapters build those foundations. The middle chapters teach controlled discovery and proof, then follow authority through sharing, files, caches, jobs, and concurrent changes. The later chapters connect the model to frameworks, managed data, architecture, testing, reporting, and independent review.

What you will learn to do

You will learn to explain the difference between authentication and authorization; write a permission matrix; recognize references in paths, bodies, headers, and stored messages; produce a controlled unauthorized read or reversible write proof; distinguish object, function, and property failures; and verify a repair without breaking legitimate sharing.

Advanced understanding includes recognizing intended access. A valid read share, scoped administrator, public object, or bearer capability can authorize a non-owner. Good analysis tests the real policy rather than assuming every changed ID is a vulnerability.

Your recurring guide

Harbor Desk is a fictional support application with two organizations and five dummy accounts. Its local lab compares vulnerable routes with a repaired policy. You can inspect dummy state, reset it, and repeat a proof with a fresh marker. The validation report distinguishes executed local evidence from framework, database, and cloud review guidance.

How to study

Predict each outcome before revealing its answer. After a mission, record actor, action, object, condition, intended rule, observed effect, and uncertainty. Use the revision atlas for recall and the glossary when a term interrupts understanding. Read in sequence if you are new; experienced readers can follow the roadmap to implementation topics.

The standard for success

A book cannot certify expertise or promise that every future system is covered. The useful goal is a durable method: explain, predict, prove, repair, and review. You will know you are progressing when you can apply the permission sentence to a new authorized feature and clearly state what the evidence does and does not establish.

Practice only on systems and accounts you own or are explicitly authorized to assess. The included exploitation exercises use fixed dummy records and reversible effects. Minimal controlled proof is a professional skill, not a compromise in depth.

Learning roadmap

Choose your route

Reader	Start here	Evidence to produce
Complete beginner	Chapters 1-10, then 11-20	Explain a request and build a small permission matrix
Developer	Foundations, then 21-50	Trace a business operation to its policy, effect, and tests
Authorized tester	Foundations, 11-30, then 51-58	Controlled proof, bounded impact, and a reproducible retest
Architect or reviewer	Policy model, lifecycle, 31-48, then capstone	Consumer map, delegation model, and consistency contract

Six stages

1. **Model, Chapters 1-10.** Learn objects, sessions, tenants, relationships, queries, roles, and failure categories. Gate: explain why knowing an ID does not grant an action.

2. **Evidence, Chapters 11-20.** Inventory routes and permissions. Use controlled accounts. Prove reads and reversible changes. Cover nested, list, bulk, and file paths. Gate: write a four-step proof with controls.

3. **Lifecycle, Chapters 21-30.** Follow tenant scope, properties, state, sharing, capabilities, caches, jobs, and races. Gate: explain what happens when a grant is revoked between steps.

4. **Modern stacks, Chapters 31-40.** Review GraphQL, REST variants, streams, services, storage, RLS, managed data, and frontend/backend boundaries. Gate: identify the actual enforcement path and unexecuted deployment tests.

5. **Repair, Chapters 41-50.** Connect framework hooks, central policy, scoped effects, output projection, and regression testing. Gate: preserve both legitimate use and denied boundaries.

6. **Judgment, Chapters 51-60.** Use tools carefully, report evidence, read advisories, respond, solve the capstone, and transfer the model. Gate: review a new authorized feature independently.

A flexible study rhythm

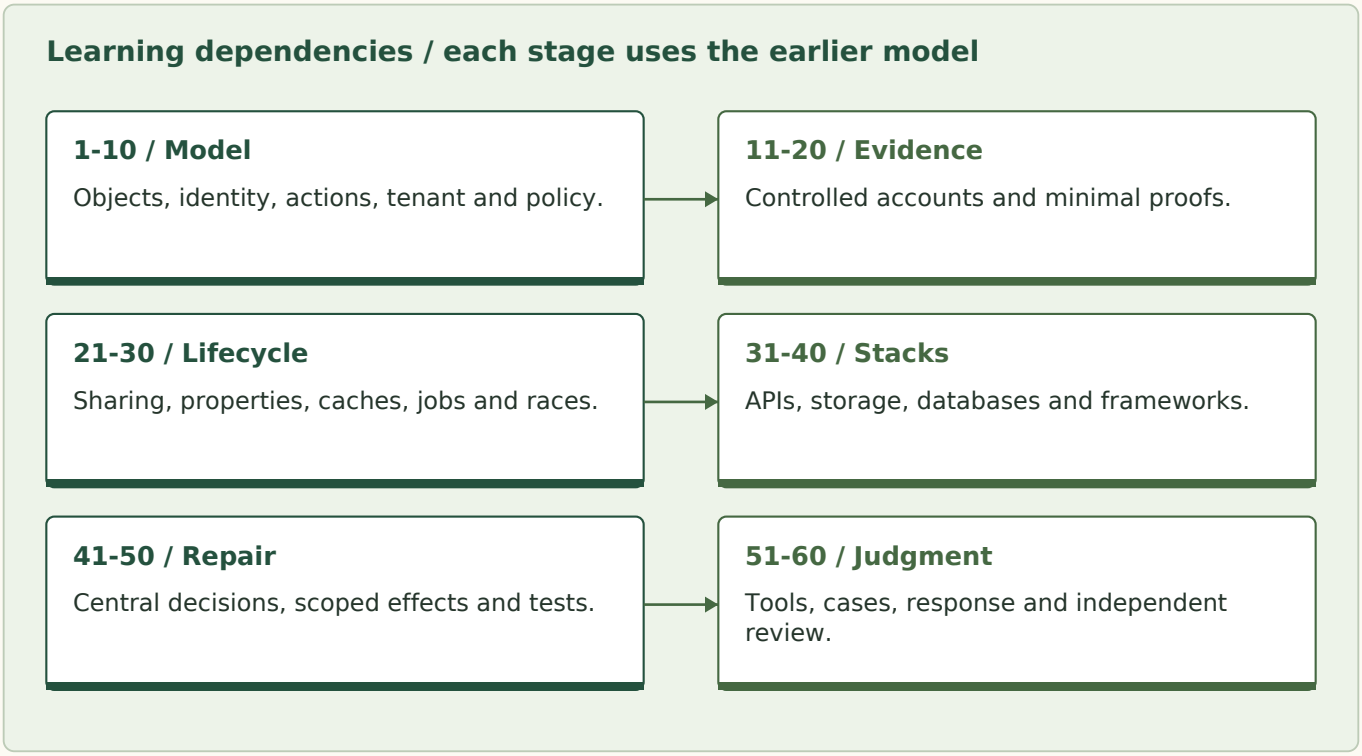
For each session: read one connected group, predict two outcomes, perform one mission or paper exercise, and write one sentence explaining the rule. End by naming one uncertainty. Time estimates vary with prior knowledge; use evidence gates rather than a fixed promise of mastery in a number of days.

For spaced review, revisit a concept after a day, a week, and a month. Reconstruct a diagram without looking, then check it. When a prediction fails, identify which input you overlooked: actor, action, resource, tenant, relationship, state, field, or time.

Where the companion files fit

The prediction deck builds reasoning before sending. The 32 lab missions connect the model to visible effects. The 24-case clinic trains classification and false-positive judgment. The notebook and report template preserve evidence. The regression matrix and mastery tracker carry the method into a new authorized system.

The preview contains the introduction, this roadmap, and the first three chapters. The paid publication contains the complete sequence, answers, appendices, workbooks, and lab. Seller materials remain outside the customer download so readers receive a clear learning package.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

I. BUILD THE PERMISSION MODEL

01. Data, objects, and references

Learn to: Tell an object from its identifier and from permission to use it.

Think of it this way: A parcel number locates a parcel. It does not make you its owner.

01A / A reference is not a permission

Object

Ticket t-101 has an owner and tenant.

Reference

t-101 tells the server which ticket.

Identity

The session says who is asking.

Decision

Policy must permit this action now.

These are distinct facts. No arrow from knowing a reference to receiving permission is valid.

Begin with one record

Data is information. A web application keeps data so it can remember useful things: a note, an order, a photo, or a support ticket. An **object** is a thing the application treats as one resource. It may be one database row, several joined rows, a file, or a result assembled on demand. An object is a business idea, not necessarily a programming-language object.

A **reference** identifies which object is wanted. In `/tickets/t-101`, `t-101` is a reference. A reference may also appear in a form, a JSON body, a cookie, a message, or a stored job. It may be a number, a random string, or a long signed token. Do not equate object references with URL parameters alone.

An **identifier** distinguishes objects within a namespace: a place where names have meaning. Ticket `101` in Cedar and ticket `101` in Maple might be different objects. A global ticket ID avoids that naming collision. It does not automatically settle which organization may use the ticket.

Three questions that must stay separate

Question	Example	What it establishes
Does the object exist?	The database finds t-101.	There is a matching record.
Who is asking?	The session belongs to Alice.	A trusted user context.
Is this action allowed?	Alice may read t-101.	Permission for this operation now.

IDOR means Insecure Direct Object Reference. It is a security weakness where a controllable reference leads to an object operation without the required authorization. The dangerous step is the missing or wrong permission decision. Merely seeing or changing an ID is not enough to prove a vulnerability.

Our running story

Harbor Desk is a fictional support system. Cedar and Maple are separate customer organizations. Alice and Bob work at Cedar. Cara works at Maple. Alice owns ticket t-101. Bob owns t-102. Cara owns t-201. These are dummy records in the bundled local lab.

If Alice requests Bob’s private ticket, the server must evaluate the policy. If Bob deliberately shared it with Alice, a read may be correct. Without that grant, returning Bob’s private body is a failure. The same bytes have different meaning under different business rules. This is why professionals begin with the permission model.

Try the model before the tools

Draw a ticket card. Put its ID on the front. Write owner, tenant, sharing grants, and state on the back. Now imagine handing someone the front. Which facts still need checking before showing the back? This short exercise separates locating from authorizing.

Misconception: “Replace the numbers with UUIDs and the issue disappears.” Random IDs reduce guessing. A leaked, shared, logged, or otherwise known ID must still meet the same permission rule.

Remember: an ID answers “which thing?” Authorization answers “may this actor perform this action on that thing?” The next chapters build the actors and decisions needed to answer that second question.

Key takeaway. Locate the object, establish the subject, and decide permission as separate steps.

Check your understanding

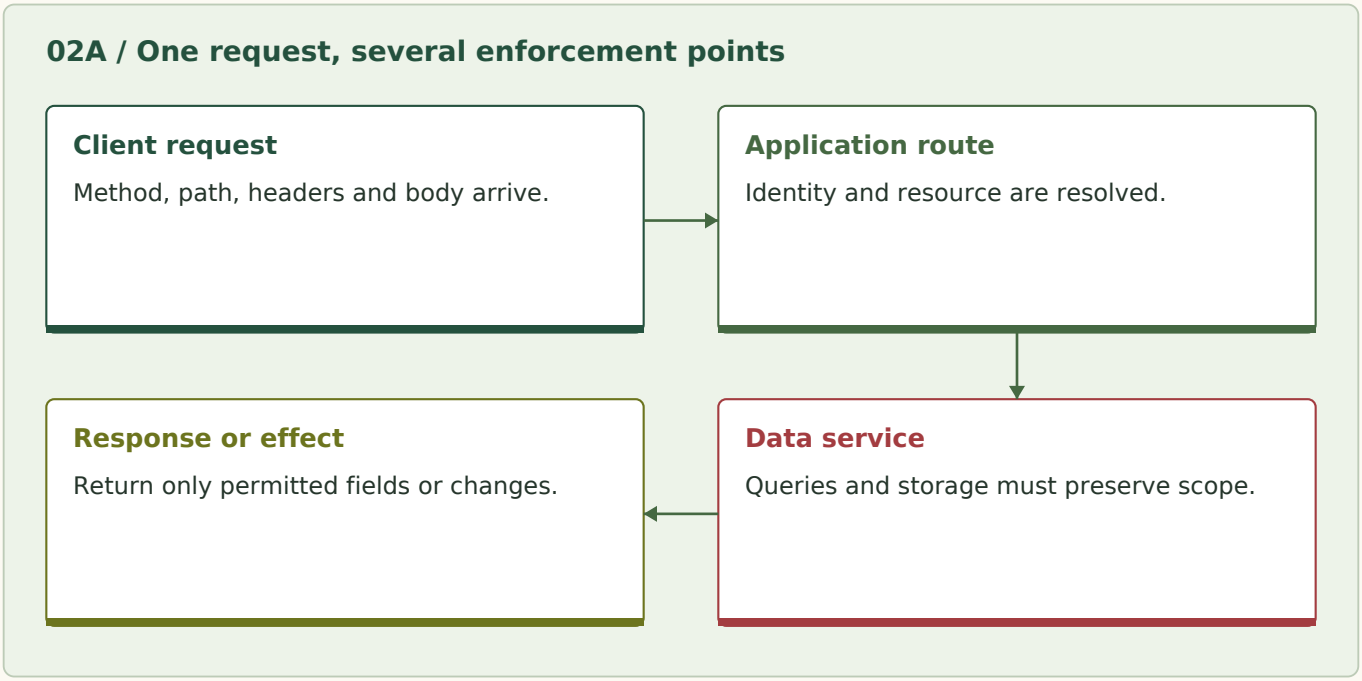
1.1 Does a visible ID prove IDOR?	**1.2** Can an object be more than one row?	**1.3** Why can a shared ticket be readable?

Answer explanations are in the full ebook. Sources: IDOR, CWE, MDN

02. How a web request reaches an object

Learn to: Read a URL, a request, and a response without confusing browser controls with server policy.

Think of it this way: A service desk receives a form; the form’s route does not prove the sender may open the requested folder.



The application and downstream data services must preserve the same permission rule.

The two sides of a web application

A **client** asks for a service. A **server** receives the request and performs work. A browser is one client. A mobile app, command-line tool, or another service can also be a client. The server may call a database or storage service before replying.

The internet moves data between networks. DNS maps a hostname to addressing information. HTTPS protects the connection using TLS. These mechanisms help data reach an intended service securely. They do not determine whether Alice may read Bob’s ticket once the request arrives.

A URL such as `https://desk.example.test:443/api/tickets/t-101?view=summary` contains a scheme, hostname, optional port, path, and query. The path selects a route and object. The query asks for a view. The fragment after #, if present, is not sent in the HTTP request target by the browser. Client code can still read a fragment and place its value into a later request.

Read this message

```
GET /api/tickets/t-101 HTTP/1.1
Host: desk.example.test
Cookie: session=opaque-example
Accept: application/json
```

The **method** is GET. The request target names a ticket. Headers supply metadata. A request body, often JSON, may carry more references. A response has a status code, headers, and possibly a body. For example, `200` with JSON says the request succeeded according to the server's response. It does not tell you whether the success was authorized.

The browser's developer tools expose requests from your own session. The Network panel helps find the method, references, and response. A request replay tool can resend them. Neither the browser nor the replay tool supplies the application's intended permission policy. You must establish that separately.

Why changing the page is not the test

A hidden Delete button is a user-interface choice. A user can still send a DELETE request directly if the server accepts it. A disabled form field can still be edited in a request. A mobile app does not make an endpoint private simply because the normal web page never calls it.

Worked example: Alice's page requests `/api/tickets/t-101`. Record that successful baseline. Use Bob's controlled account to request that same ID. Keep the route and method fixed. If the body is private, Bob has no grant, and Bob receives Alice's unique dummy marker, you have evidence of an unauthorized read. Later chapters add negative controls and side-effect checks.

Practice

Write down the client, server, requested object, and operation for a download. Add storage as a separate component if the server redirects to it. Mark each place that can return the bytes. Every such place needs the relevant permission contract.

Common mistake: believing HTTPS, CORS, a mobile client, or an unlisted route is object authorization. Each serves another purpose. A server must make its own permission decision for every protected operation.

Key takeaway. A request is an instruction to the server. The server must authorize it independently of the interface.

Check your understanding

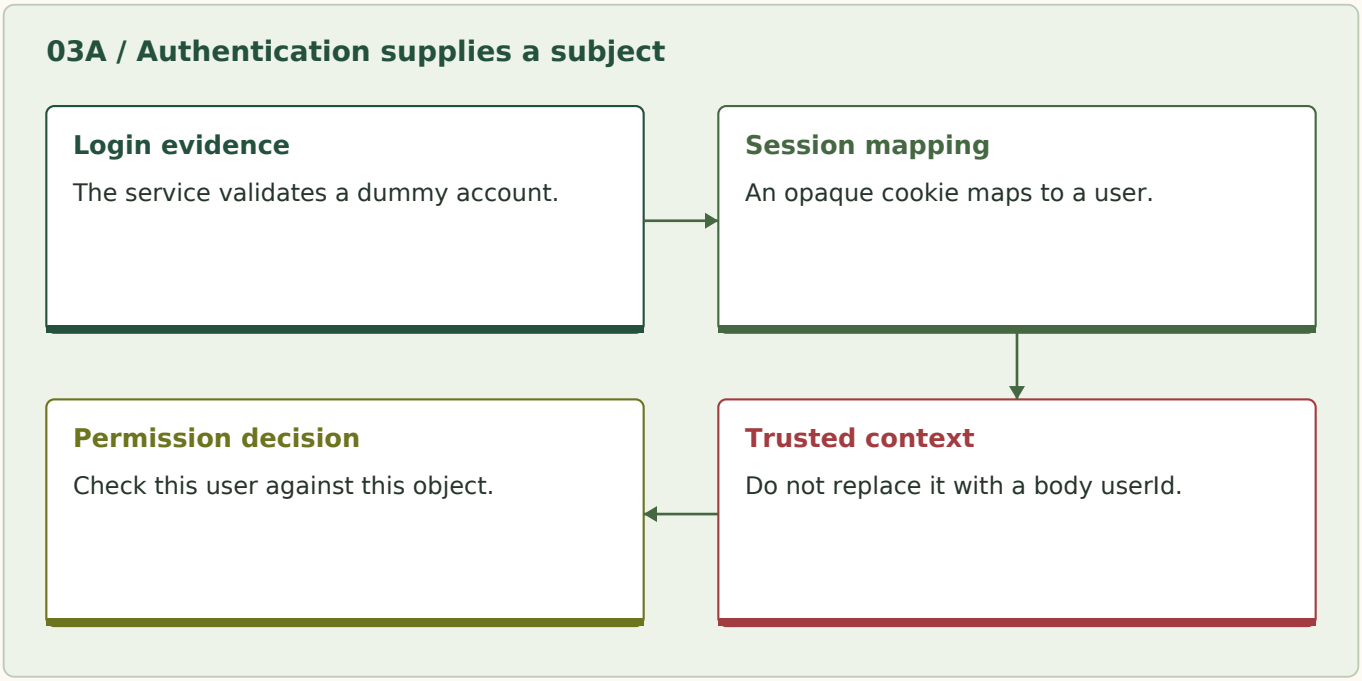
2.1 Does HTTPS authorize ticket access?	**2.2** Where else can an ID appear?	**2.3** Why is a hidden button insufficient?

Answer explanations are in the full ebook. Sources: HTTP, IDOR, WSTG

03. Identity, authentication, and sessions

Learn to: Understand where the trusted subject comes from and why login does not authorize every object.

Think of it this way: A building badge identifies a person. It does not unlock every room.



A valid session is the starting point for authorization, not its conclusion.

Identity comes before permission

An **identity** names an actor. **Authentication** checks evidence that the actor is who they claim to be. A password, a passkey, or a valid access token may be part of that process. **Authorization** decides what that authenticated actor may do. A system can authenticate Alice correctly and still authorize her incorrectly.

HTTP requests do not automatically remember a prior login. A **session** gives requests continuing context. In a common design, the server creates a random session identifier after login, stores its association with a user, and puts the identifier in a browser cookie. On a later request, the browser sends the cookie when its rules allow. The server looks up the session and builds a trusted user context.

The cookie is not the user record. The session identifier should be hard to guess and protected from theft. `HttpOnly` limits ordinary script access to the cookie; `Secure` restricts transport to HTTPS; `SameSite` changes cross-site delivery. None of these flags says that a ticket belongs to the current user.

Do not accept a self-declared subject

This is unsafe when the caller controls `userId`:

```
subject = request.body.userId
return allTicketsOwnedBy(subject)
```

An authenticated request can still contain untrusted identity fields. Use the validated session or token to establish the actor. A request may intentionally name another user for an authorized admin operation. In that case, keep **actor** and **target user** separate, and authorize the delegation explicitly.

A JWT is a token format. A valid signature establishes that the token came from the expected issuer under the accepted validation rules. The service must also check intended audience, expiry, and other required claims. Even a correctly validated token may contain a role that became stale after a membership change. Current object permission is a further decision, not a consequence of the word JWT.

Lab identities

Harbor Desk uses real random session cookies for known dummy accounts. Alice, Bob, Cara, Dana, and Quinn have different fixture permissions. The server derives their identity from the session. The lab's control key protects local training controls. It does not authorize business objects.

Two tabs in one browser profile normally share the same cookie jar. Logging in as Bob in one tab can change Alice's apparent session in another. Use separate browser profiles or isolated contexts for simultaneous accounts, or switch accounts deliberately and confirm the displayed subject before each request. A mistaken account makes a convincing-looking comparison meaningless.

Check your reasoning

Trace login, cookie storage, cookie delivery, session lookup, and permission decision. Then imagine a valid Alice session requesting Cara's ticket. Which earlier steps can succeed while the final decision must deny? All authentication steps can succeed.

Misconception: "It requires login, so it is secure." Login removes anonymous access. It does not prevent one valid customer from reaching another customer's records. The policy must bind the trusted subject to the action and object.

Key takeaway. Use trusted identity context, then evaluate object permission. A valid login alone is never the whole policy.

Check your understanding

3.1 Is a body userId trusted?	**3.2** Why use separate browser profiles?	**3.3** Does a signed role claim prove current permission?

Answer explanations are in the full ebook. Sources: COOKIE, JWT, AUTHZ

Sources and further reading

Research cutoff: 27 October 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

IDOR - OWASP: IDOR Prevention Cheat Sheet

Consulted. Object-level permission checks and scoped lookups. The Harbor Desk examples and teaching diagrams are original.

https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

AUTHZ - OWASP: Authorization Cheat Sheet

Consulted. Least privilege, deny by default, permission checks, and review. Business rules still need an explicit application policy.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

WSTG - OWASP WSTG: Testing for IDOR

Consulted. Controlled comparison of references and users. Latest is a living guide; record the tested application version.

https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/05-Authorization/04-Insecure_Direct_Object_References/

CWE - MITRE CWE-639: Authorization Bypass Through User-Controlled Key

Consulted. Weakness definition and terminology. Taxonomy is not a substitute for an evidence-backed report.

https://cwe.mitre.org/data/definitions/639.html

MDN - MDN: Insecure Direct Object Reference

Consulted. Educational framing of object references and server authorization.

https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/IDOR

HTTP - RFC 9110: HTTP Semantics

Targeted further reading. Methods, status codes, conditional requests and range requests. Application policy determines who may act.

https://www.rfc-editor.org/rfc/rfc9110.html

COOKIE - RFC 6265: HTTP State Management Mechanism

Targeted further reading. Cookie storage and delivery foundations; newer cookie behavior requires current browser documentation.

https://www.rfc-editor.org/rfc/rfc6265.html

JWT - RFC 8725: JSON Web Token Best Current Practices

Targeted further reading. Token validation context. Valid signatures do not decide every current object permission.

https://www.rfc-editor.org/rfc/rfc8725.html